สอน AI ให้รู้จักภาพ เสียง และท่าทาง

เอกสารการสอนสำหรับผู้เริ่มต้น เรียนจากโปรเจกต์จริงใน Teachable-Machine-AnyThing

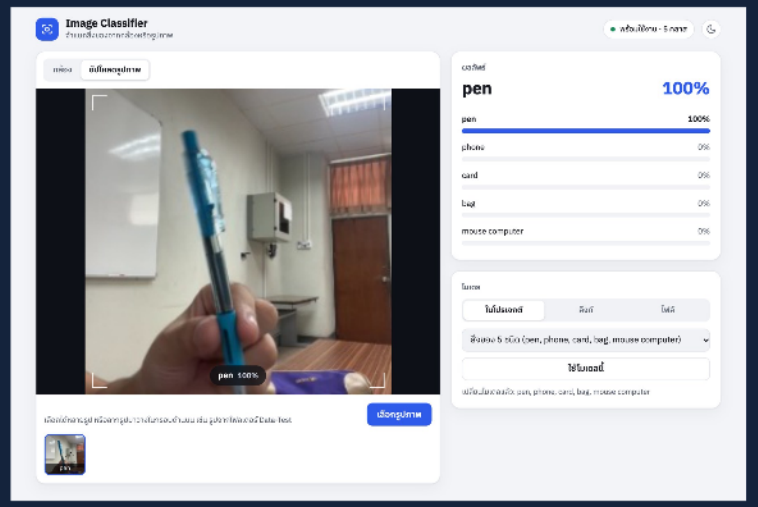
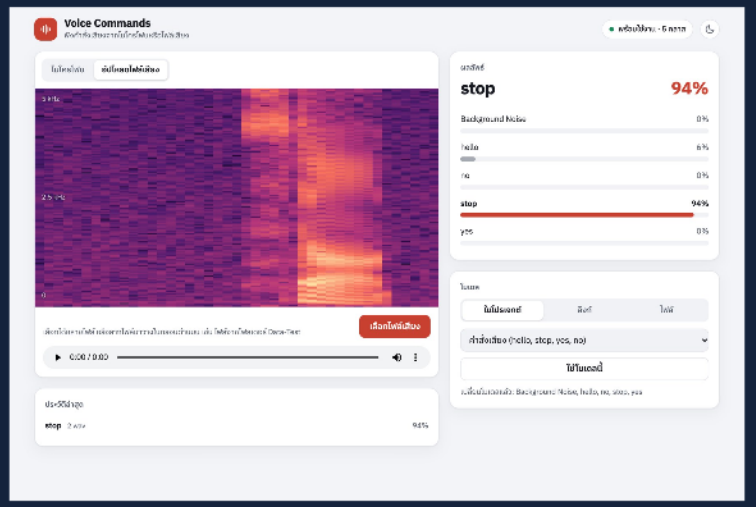
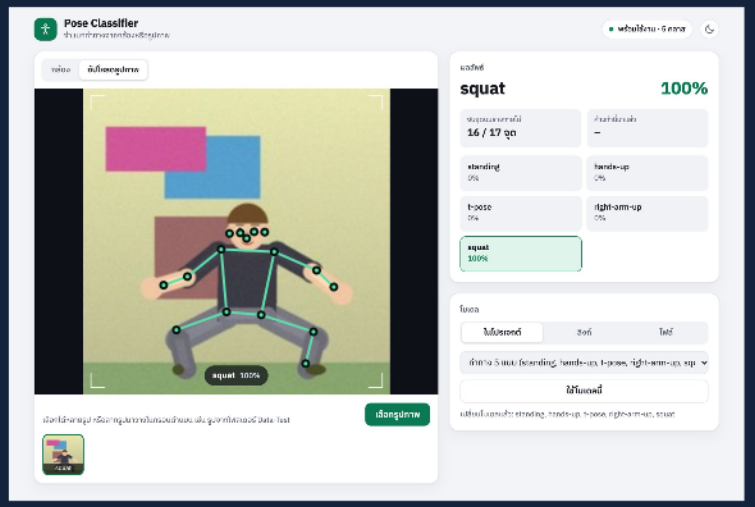


Image: จำแนกสิ่งของ



Audio: ฟังคำสั่งเสียง



Pose: จำแนกท่าทาง

ภาพหน้าจอและผลลัพธ์ในเอกสารนี้มาจากการรันจริง โดยติดป้ายบอกที่มาของข้อมูลทุกจุด

เรียนจบเอกสารนี้แล้ว คุณจะทำอะไรได้

เข้าใจ

อธิบายได้ว่าโมเดล AI คืออะไร เรียนรู้จากตัวอย่างอย่างไร และทำไมความมั่นใจ 100% จึงยังผิดได้

ลงมือทำ

เทรนโมเดลบน Teachable Machine แล้ว export มาใช้ได้ โดยไม่ต้องเขียนโค้ดตอนเทรน

นำไปใช้

เปิดแอปบนเว็บ รันโมเดลด้วย Python และต่อยอดด้วยฟังก์ชัน on_result

ประเมินผล

อ่านผลทดสอบและ confusion matrix เป็น และบอกข้อจำกัดของโมเดลได้ตรงไปตรงมา

เส้นทางการเรียน 7 บท

1	2	3	4	5	6	7
AI คืออะไร	ข้อมูล	เทรนโมเดล	ข้างในโมเดล	นำไปใช้	วัดผล	ฝึกปฏิบัติ

ไม่ต้องมีพื้นฐาน AI หรือคณิตศาสตร์ ถ้าเจอคำใหม่ ให้ดูอภิธานศัพท์ท้ายเล่ม

ก่อนเริ่ม: เตรียมของ 4 อย่าง

สิ่งที่ต้องมี	ใช้ทำอะไร	หมายเหตุ
เบราว์เซอร์	เปิด Teachable Machine และแอปบนเว็บ	เอกสารนี้ทดสอบด้วย Google Chrome
อินเทอร์เน็ต	เทรนบนเว็บ และโหลดไลบรารีของแอป	เว็บแอปยังทำงานแบบออฟไลน์ไม่ได้
Python 3.9 ขึ้นไป	เปิดเว็บแอปในเครื่อง และรัน run_model.py	ทดสอบแล้วกับ 3.9 และ 3.10 บน macOS
โค้ดจาก GitHub	ข้อมูล โมเดล แอป และสคริปต์ครบในที่เดียว	ติดตั้งตามขั้นตอนในบทที่ 5

ป้ายบอกที่มาของข้อมูล

ข้อมูลจริง รูปถ่ายจริงของ Image ข้อมูลจำลอง เสียงของ Audio และภาพคนของ Pose สร้างด้วยโปรแกรม

เราไม่นำภาพอื่นมาแทน และจะติดป้ายนี้กำกับ เพื่อไม่ให้เข้าใจผิดว่าโมเดล Audio และ Pose ผ่านการทดสอบกับของจริงแล้ว

บทที่ 1

AI คืออะไร

AI ในเอกสารนี้ไม่ได้ถูกสั่งด้วยกฎทีละข้อ แต่เรียนรู้จากตัวอย่างที่เราให้

โปรแกรมทั่วไปทำตามกฎ แต่โมเดลเรียนรู้จากตัวอย่าง

โปรแกรมทั่วไป

ลองเขียนกฎแยก “ปากกา” จากพิกเซลดูสิ เงา มุมกล้อง และสีที่เปลี่ยนไป ทำให้กฎยาวไม่รู้จบ

แมชชีนเลิร์นนิง

เราให้รูปปากกา 200 รูปพร้อมบอกว่านี่คือ pen แล้วให้คอมพิวเตอร์หารูปแบบเอง ผลที่ได้คือโมเดลที่ใช้ทายรูปใหม่ได้

โปรเจกต์นี้สอน AI 3 งาน ด้วยโครงสร้างเดียวกัน

งาน	AI ทำอะไร	คลาส (คำตอบที่เป็นไปได้)	ข้อมูลเทรน	ที่มา
Image	ดูรูปแล้วบอกว่าเป็นของอะไร	pen, phone, card, bag, mouse computer	คลาสละ 200 รูป	รูปถ่ายจริง
Audio	ฟังเสียง 1 วินาทีแล้วบอกคำ	Background Noise, hello, stop, yes, no	คลาสละ 60 คลิป	สังเคราะห์
Pose	ดูรูปคนแล้วบอกท่าทาง	standing, hands-up, t-pose, right-arm-up, squat	คลาสละ 100 รูป	ภาพจำลอง

ทุกโปรเจกต์มีโฟลเดอร์เหมือนกัน

`Data-Training`
ข้อมูลที่ใช้สอน

`Data-Test`
ข้อมูลไว้สอบ

`project.tm`
เปิดเทรนต่อบนเว็บ

`Model`
โมเดลที่ export มา

`application`
แอปและ Python

คำศัพท์ 6 คำที่ใช้ตลอดทั้งเอกสาร

ตัวอย่าง (Sample)

ข้อมูลหนึ่งชิ้นที่ใช้สอน เช่น รูปปากกา 1 รูป หรือเสียง hello 1 คลิป

คลาส (Class)

คำตอบที่โมเดลเลือกได้ ถ้าไม่มีคลาสนั้น โมเดลก็จะตอบคำนั้นไม่ได้

โมเดล (Model)

ไฟล์ที่เก็บตัวเลขจำนวนมาก ซึ่งได้จากการเรียนรู้ตัวอย่าง

การเทรน (Training)

กระบวนการปรับตัวเลขในโมเดลซ้ำ ๆ ให้ตอบถูกมากขึ้น

การทำนาย (Prediction)

การนำโมเดลไปตอบข้อมูลใหม่ที่ไม่เคยเห็นตอนเรียน

ความมั่นใจ (Confidence)

ค่าที่โมเดลให้แต่ละคลาส รวมกันได้ 100% ไม่ได้แปลว่าถูกเสมอ

งาน AI ทุกครั้งเดินตามวงจร 5 ขั้น

1
เก็บข้อมูล
ถ่ายรูปหรืออัดเสียง แยกตามคลาส

→

2
เทรน
ให้โมเดลเรียนจากตัวอย่าง

→

3
ทดสอบ
ลองกับข้อมูลที่ไม่เคยใช้สอน

→

4
Export
ดาวน์โหลดไฟล์โมเดล

→

5
ใช้งาน
ใส่ในแอปหรือ Python

ทำไมต้องวนกลับ

ถ้าผลทดสอบแย่ คำตอบมักไม่ใช่การแก้โค้ด แต่ต้องกลับไปดูข้อมูล
ตัวอย่างจริงในบทที่ 6: โมเดล Pose ทาย right-arm-up ผิด 2 จาก 2 รูป ข้อแนะนำจึงเป็นการเพิ่มตัวอย่างในคลาสนี้ก่อน ไม่ใช่การเปลี่ยนแอป
ในเอกสารนี้ เราจะเดินตามวงจรนี้ครบทุกขั้นด้วยโปรเจกต์จริง

บทที่ 2

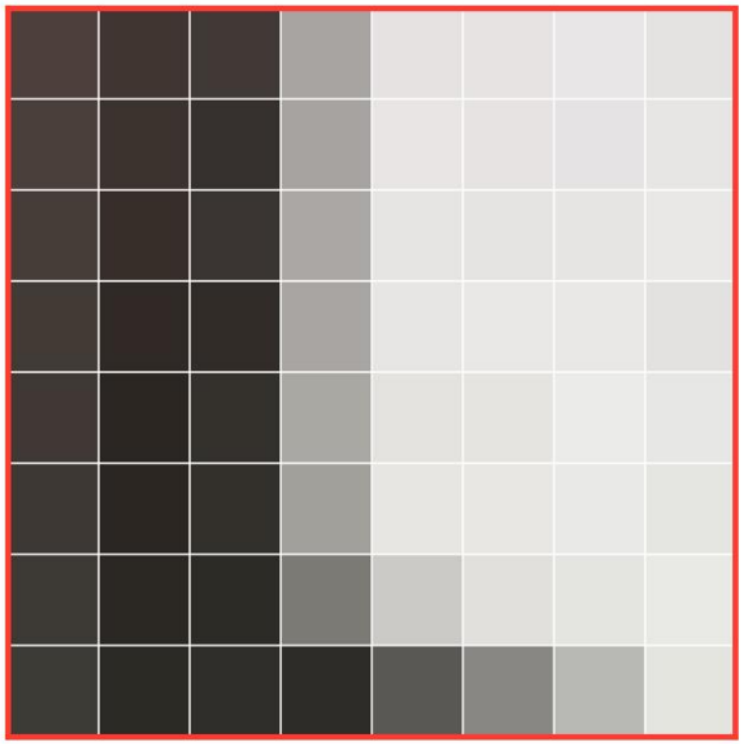
ข้อมูล: สิ่งที่โมเดลเรียน

คอมพิวเตอร์ไม่เห็นปากกาหรือได้ยินเสียง มันเห็นแต่ตัวเลข เราจะดูว่าตัวเลขเหล่านั้นมาจากไหน

คอมพิวเตอร์เห็นรูปเป็นตารางตัวเลข

รูปเต็ม 224 × 224 พิกเซล

ซูมกรอบแดง 8 × 8 พิกเซล



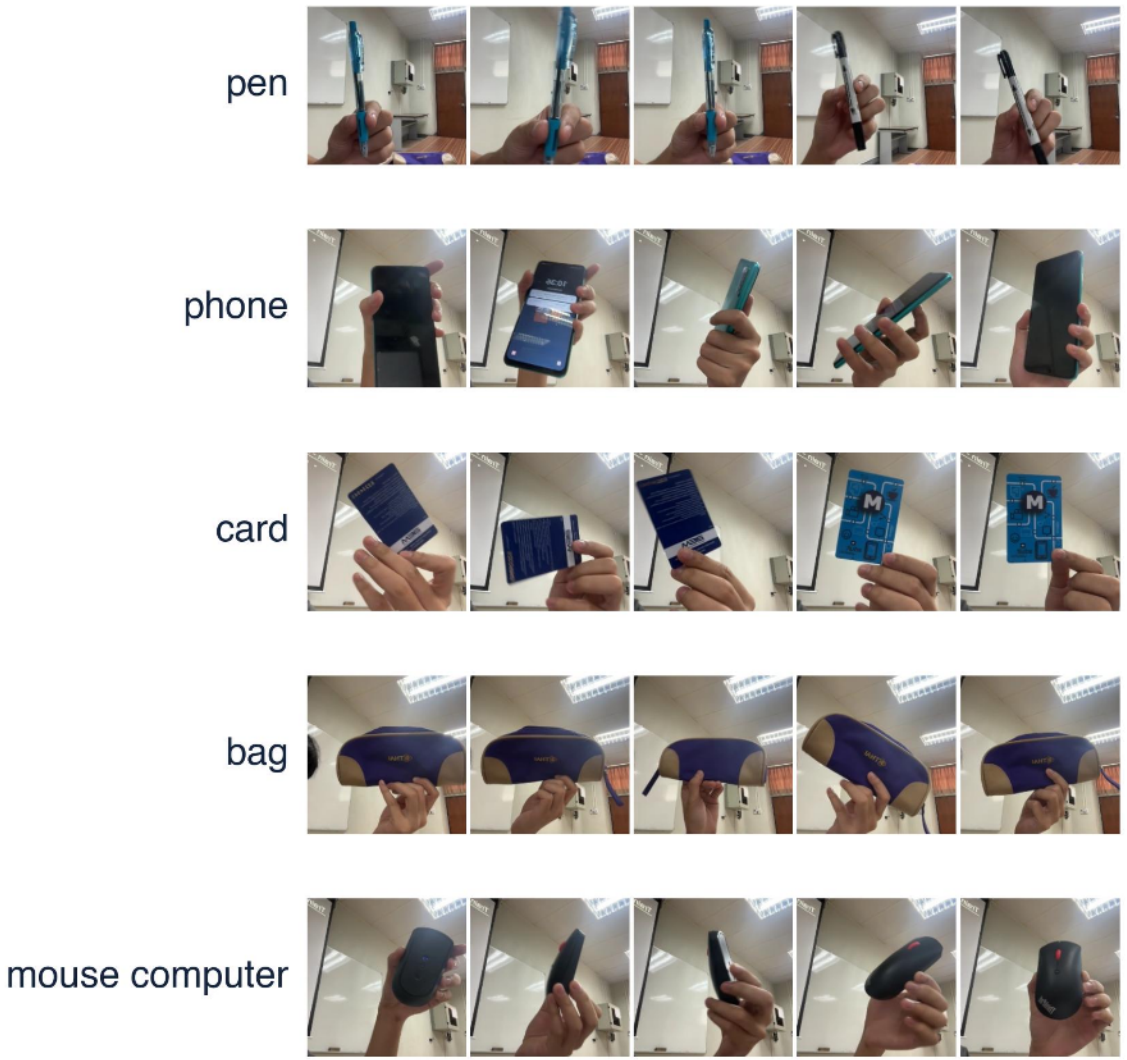
พิกเซลเดียวกันเมื่อเป็นตัวเลข (R, G, B)

R 77 G 63 B 60	R 64 G 53 B 49	R 65 G 57 B 54
R 74 G 63 B 59	R 60 G 51 B 46	R 54 G 49 B 46
R 70 G 61 B 56	R 55 G 46 B 41	R 58 G 53 B 50
R 66 G 59 B 53	R 49 G 41 B 38	R 49 G 44 B 40

ข้อมูลจริง Image-Project/Data-Test/3.jpg ค่าสีอ่านจากไฟล์จริง

รูปนี้มี 224 × 224 = 50,176 พิกเซล แต่ละพิกเซลมีเลข 3 ตัว (แดง เขียว น้ำเงิน) รวมเป็นตัวเลข 150,528 ค่า ก่อนเข้าโมเดล โค้ดแปลงทุกค่าด้วยสูตร ค่า/127 - 1 เพื่อให้อยู่ในช่วงประมาณ -1 ถึง 1

ชุดข้อมูล Image: รูปถ่ายจริง คลาสละ 200 รูป



ข้อมูลจริง ตัวอย่างจาก Image-Project/Data-Training

สังเกตอะไรจากรูปชุดนี้

- มือถือของจากหลายมุม ทำให้โมเดลเห็นวัตถุเดียวกันในหลายรูปแบบ
- คลาส card มีภาพการ์ดลวดลายต่างกัน ซึ่งช่วยให้โมเดลเห็นรูปแบบกว้างขึ้น
- ทุกรูปมีขนาด 224 × 224 พิกเซล เท่ากับขนาดที่โมเดลรับ

ข้อควรระวัง

รูปในภาพนี้ถ่ายในห้องเดียวกัน พื้นหลังและมือจึงคล้ายกันมาก โมเดลอาจเก่งเฉพาะฉากแบบนี้ ถ้านำไปใช้ห้องอื่นหรือแสงอื่น ต้องทดสอบเพิ่มก่อนเชื่อผล เราจะกลับมาที่เรื่องนี้ในบทที่ 3

แบ่งข้อมูลเป็น 2 กอง: กองเรียนกับกองสอบ

Training 85% = 170 รูปต่อคลาส
ใช้สอนโมเดล

Test 15%
30 รูป

การแบ่งของ Teachable Machine เมื่อแต่ละคลาสมี 200 รูป หน้าเว็บแสดง 30 รูปต่อคลาสสำหรับการวัดผล

โปรเจกต์	ไฟล์ใน Data-Test
Image	5 รูป
Audio	10 ไฟล์เสียง
Pose	10 รูป

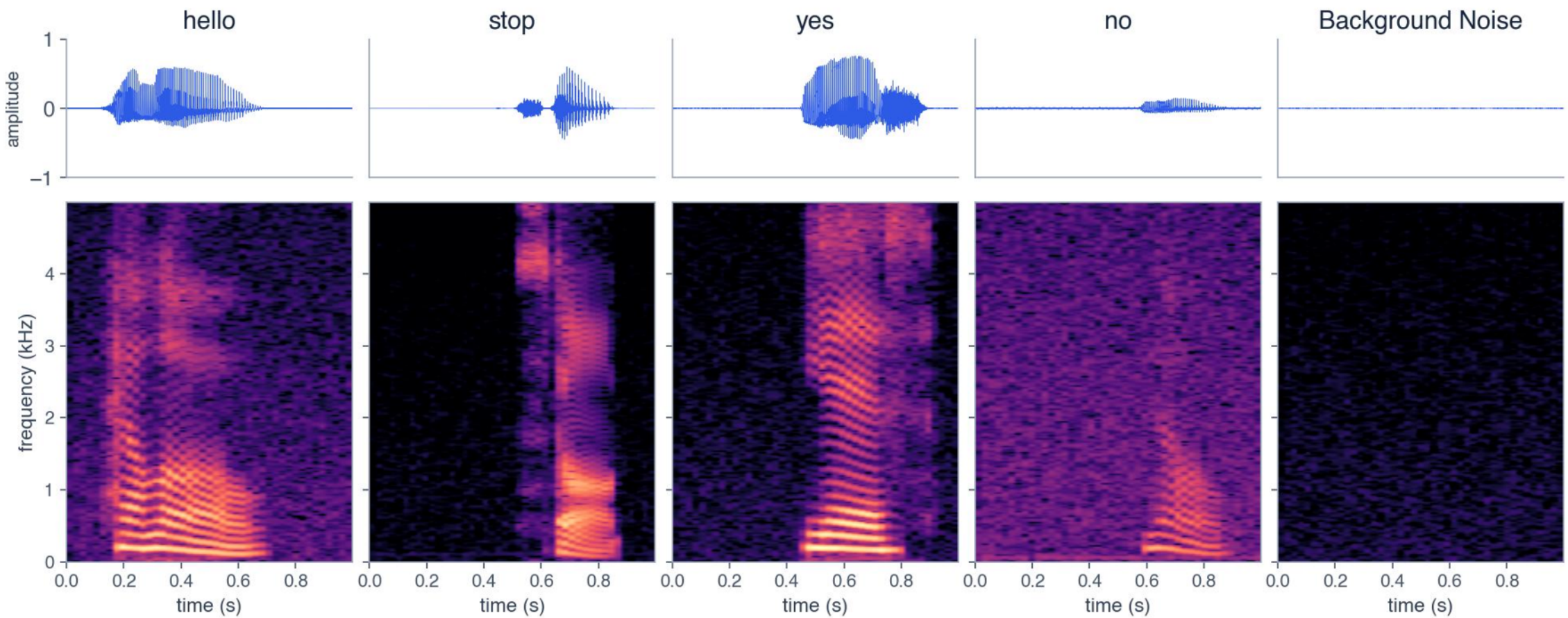
ทำไมต้องแยก

- เหมือนข้อสอบที่ไม่ควรเฉลยให้ดูก่อน ถ้าโมเดลเคยเห็นคำตอบแล้ว คะแนนสูงก็ไม่ได้บอกว่าเก่งจริง
- Teachable Machine แบ่งกองนี้ให้เองอัตโนมัติ เราไม่ต้องทำเอง
- Repo นี้ยังมี Data-Test แยกอีกชุด สำหรับสอบหลังจากได้โมเดลแล้ว

ตัวอย่าง Data-Test ของ Image

ซ้ายไปขวา: card, mouse computer, pen, phone และ bag โมเดลทายถูกทั้ง 5 รูปด้วย run_model.py

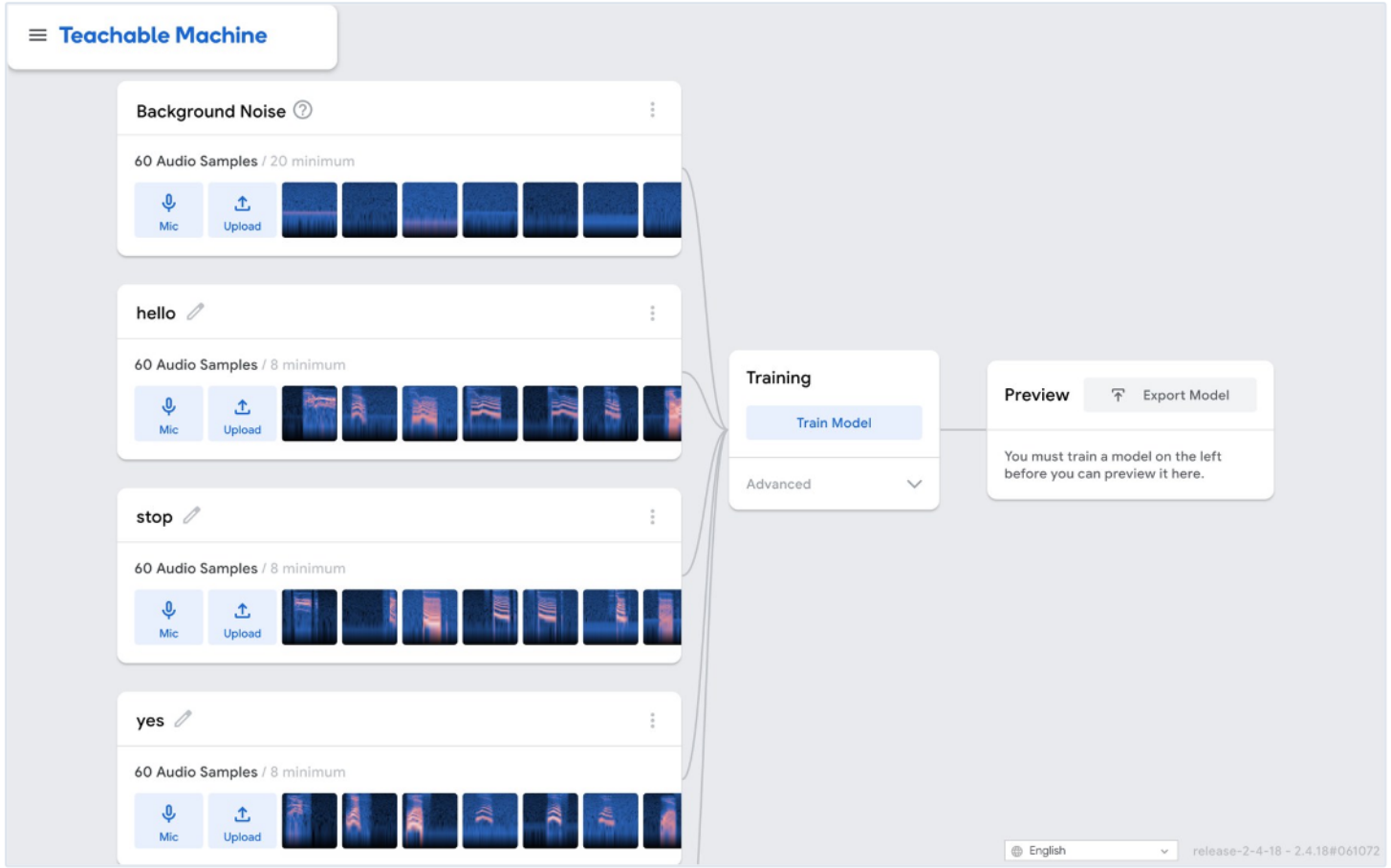
เสียงกลายเป็นภาพได้ด้วยสเปกโตรแกรม



ข้อมูลจำลอง Audio-Project/Data-Test ไฟล์ 1–5 เป็นเสียงสังเคราะห์ คำนวณด้วยโค้ดของ run_model.py

แกนนอนคือเวลา แกนตั้งคือความถี่ สีสว่างคือเสียงดัง ไฟล์ 1 วินาทีกลายเป็นตาราง 43 × 232 ช่อง โมเดลจึงอ่านเสียงได้แบบเดียวกับอ่านรูป ภาพ Background Noise มืดเพราะไม่มีเสียงดัง

ชุดข้อมูล Audio บน Teachable Machine



ภาพหน้าจอจริง ข้อมูลจำลอง

แต่ละคลาสมี 60 ตัวอย่าง

- เว็บแสดงเสียงเป็นภาพสเปกโตรแกรมโดยตรง
- จำนวนขั้นต่ำที่เว็บกำหนด: Background Noise 20 ตัวอย่าง คลาสอื่น 8 ตัวอย่าง
- ตัวอย่างแต่ละชิ้นยาว 1 วินาที
- การอัปโหลดเสียงต้องใช้ไฟล์ .zip จึงมี <คลาส>-samples.zip ใน Data-Training

ทำไมต้องมี Background Noise

เพื่อให้โมเดลมีคำตอบว่า “ไม่มีคำสั่ง” มิฉะนั้นมันต้องเลือกคำใดคำหนึ่งเสมอ

ชุดข้อมูล Pose: โมเดลดูตำแหน่งร่างกาย

ข้อมูลจำลอง ตัวอย่างจาก Pose-Project/Data-Training

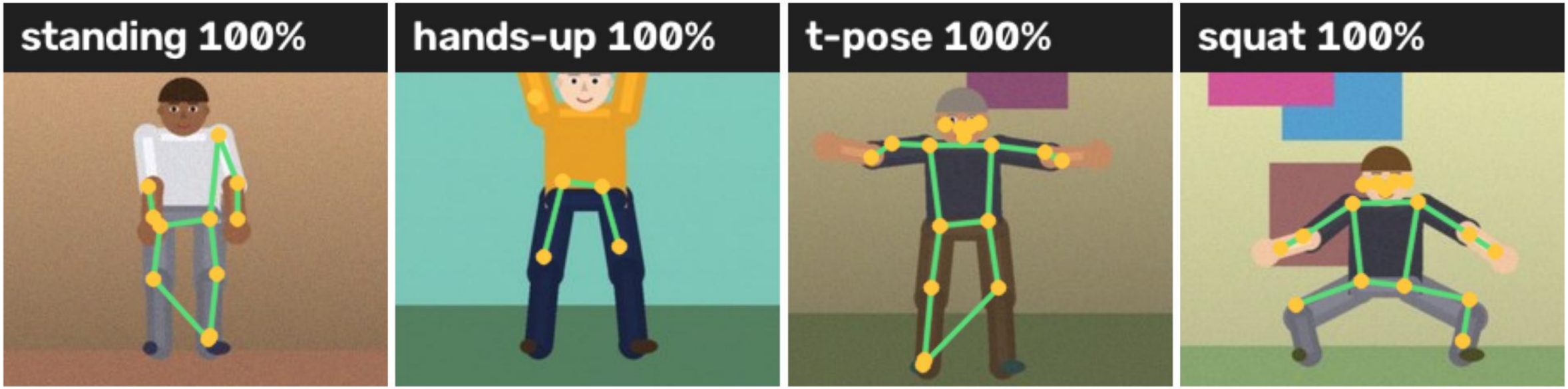
ภาพคนที่สร้างด้วยโปรแกรม

- คลาสละ 100 รูป สุ่มสีเสื้อ สีผิว พื้นหลัง และตำแหน่ง
- จุดประสงค์คือเรียนรู้ขั้นตอนการทำงาน ไม่ใช่เตรียมไว้ใช้กับคนจริง
- โมเดลไม่ได้เรียนจากรูปตรง ๆ แต่เรียนจากข้อมูลตำแหน่งที่ PoseNet ส่งให้

ก่อนนำไปใช้กับคนจริง

ผลทดสอบในเอกสารนี้วัดกับภาพจำลองเท่านั้น ถ้าจะใช้กับคนจริง ให้เปิด project.tm แล้วเก็บตัวอย่างจากกล้องในสถานที่จริงและเทรนใหม่ก่อน

PoseNet เปลี่ยนรูปคนเป็น 17 จุด



ข้อมูลจำลอง ผลจริงจาก run_model.py บน Pose-Project/Data-Test แสดงจุดที่มั่นใจตั้งแต่ 50% ขึ้นไป บางจุดคลาดเคลื่อน เช่น เส้นไขว้ในภาพแรก

ขั้นที่ 1: PoseNet

โมเดลสำเร็จรูปหาตำแหน่งตา จมูก ไหล่ ศอก ข้อมือ สะโพก เข่า และข้อเท้า รวม 17 จุด

ขั้นที่ 2: โมเดลของเรา

รับข้อมูลตำแหน่งที่ได้ แล้วจำแนกว่าเป็นท่าใดใน 5 ท่า โดยไม่ได้เรียนสีเสื้อหรือพื้นหลังตรง ๆ

ข้อมูลที่ดีเป็นอย่างไร

หลักคิด	ทำอย่างไร	ถ้าไม่ทำจะเกิดอะไร
ตรงกับการใช้งานจริง	ถ่ายในสถานที่และแสงที่จะใช้จริง	ทดสอบได้ดี แต่ใช้จริงแล้วผิดบ่อย
หลากหลาย	เปลี่ยนมุม ระยะ มือ และพื้นหลัง	โมเดลจำฉากแทนที่จะจำวัตถุ
สมดุล	ให้จำนวนตัวอย่างแต่ละคลาสใกล้เคียงกัน	โมเดลมีแนวโน้มเอนเอียงไปทางคลาสที่มีมาก
มีคลาส “ไม่มีอะไร”	ใส่ Background Noise หรือฉากว่าง ถ้างานต้องการ	โมเดลบังคับเลือกคำตอบแม้ไม่มีสิ่งที่ต้องหา
ไม่ปนกัน	ใส่ตัวอย่างให้ถูกคลาส และแยกข้อมูลสอบ	คะแนนสูงเกินจริง หรือเรียนจากคำตอบที่ผิด

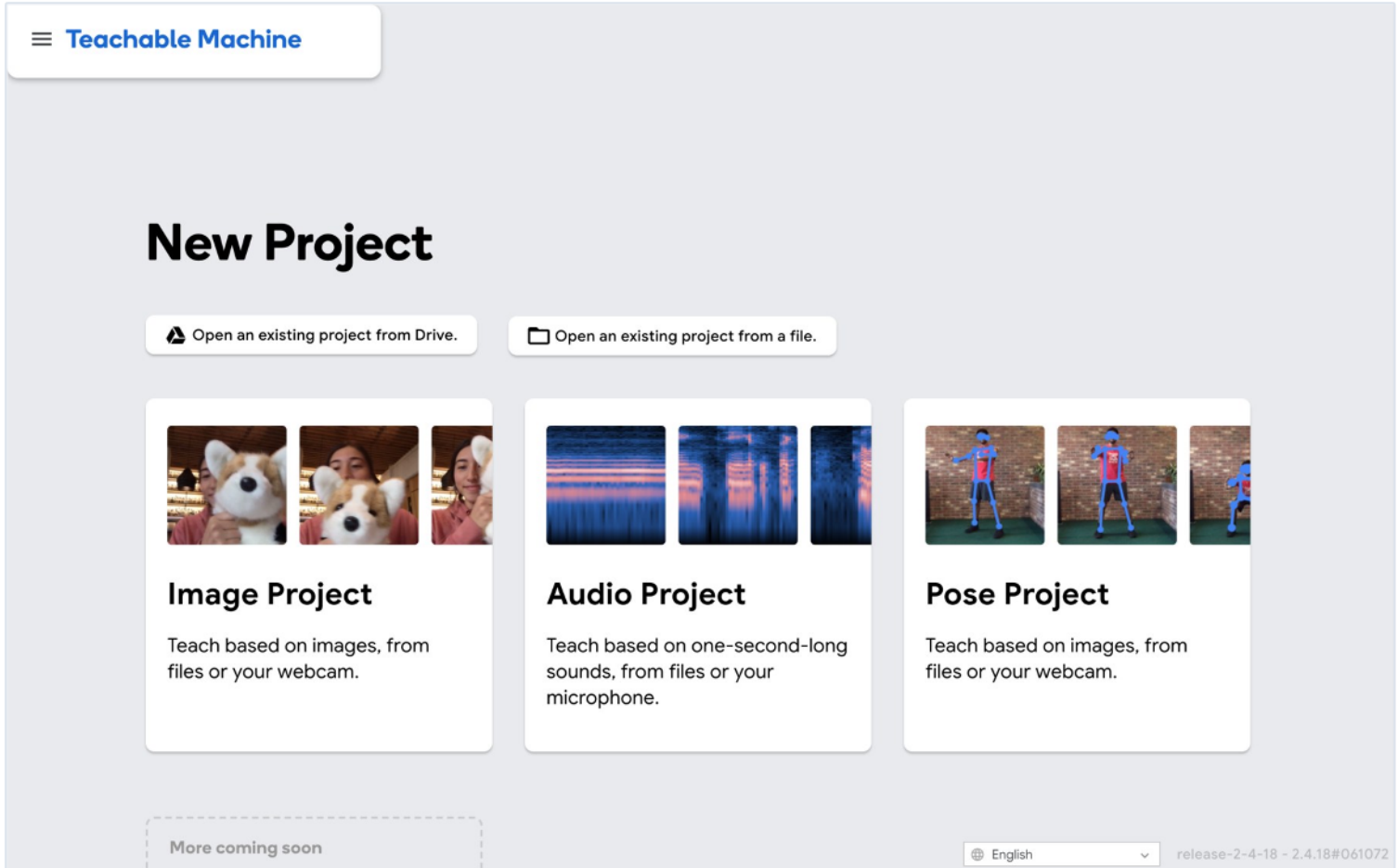
ความจริงข้อหนึ่งของงาน AI: ข้อมูลที่ดีขึ้นมักช่วยได้มากกว่าการเปลี่ยนโมเดล

บทที่ 3

เทรนโมเดลบน Teachable Machine

ทำตามทีละขั้นด้วย project.tm ของจริง และดูผลการเทรนจริงจากหน้าเว็บ

ขั้นที่ 1: เปิดหน้าสร้างโปรเจกต์



ภาพหน้าจอจริง teachablemachine.withgoogle.com/train รุ่น 2.4.18

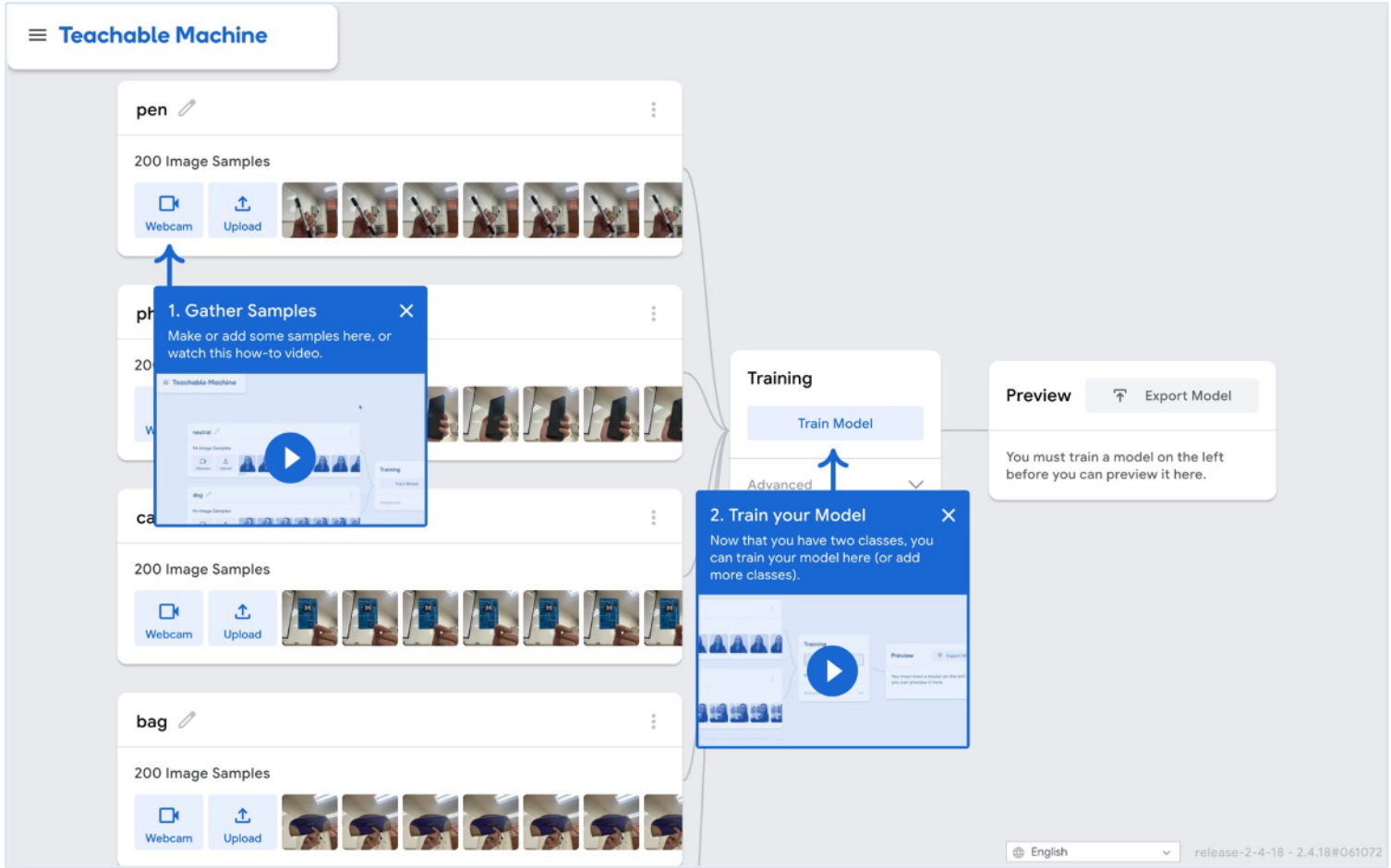
ทำตามนี้

1. เปิดเว็บ Teachable Machine แล้วกด Get Started
2. กด Open an existing project from a file
3. เลือก Image-Project/project.tm

ถ้าเริ่มงานใหม่เอง

เลือกการ์ด Image, Audio หรือ Pose แล้วเพิ่มคลาสกับตัวอย่างจากกล้อง ไมค์ หรือไฟล์ได้เลย

ขั้นที่ 2: ตรวจตัวอย่างก่อนเทรน



ภาพหน้าจอจริง เปิดจาก Image-Project/project.tm

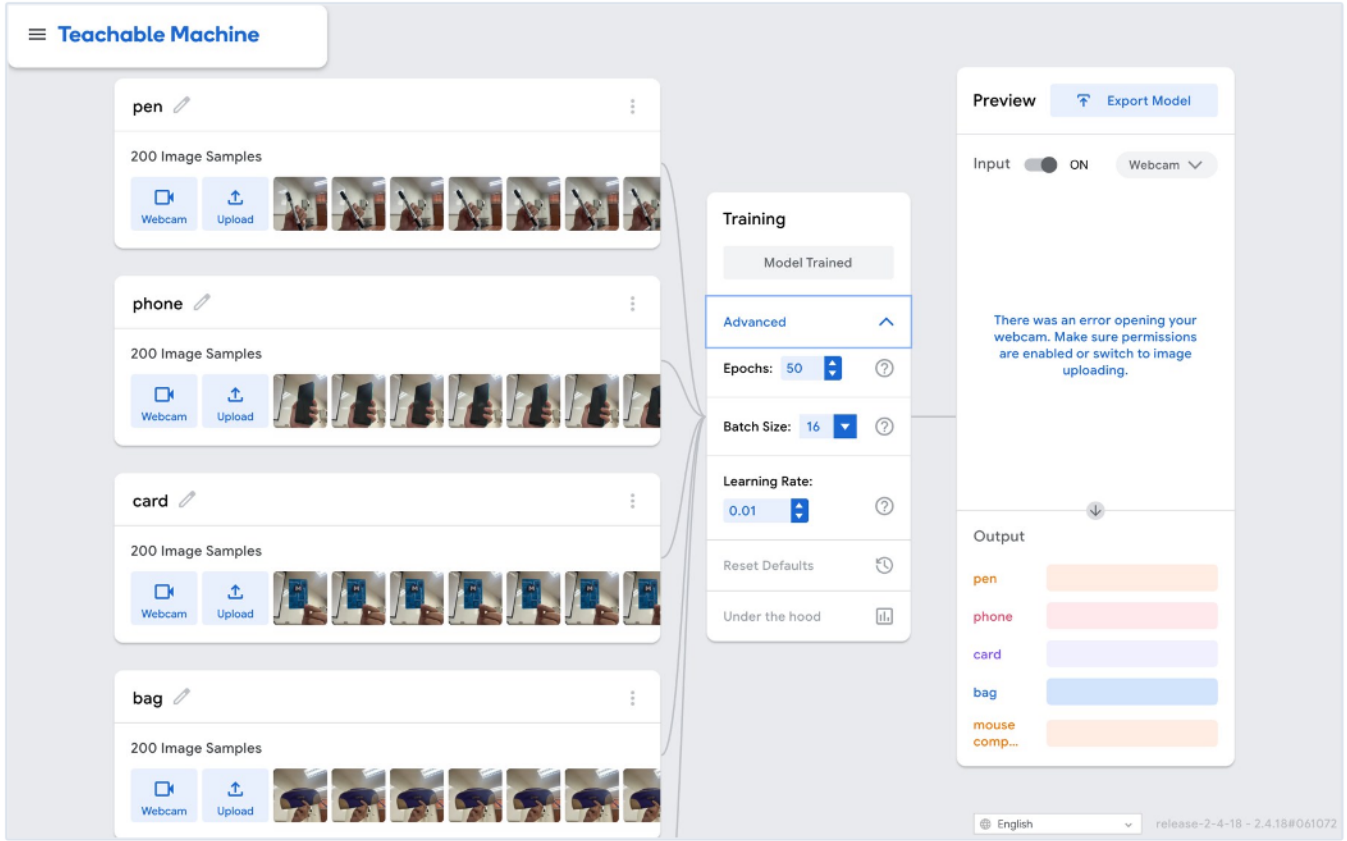
อ่านหน้านี้จากซ้ายไปขวา

- ซ้าย: คลาสและตัวอย่าง (เห็น 200 Image Samples)
- กลาง: ปุ่ม Train Model
- ขวา: Preview ทดลองโมเดล และ Export Model

ตรวจอะไรก่อนกดเทรน

ชื่อคลาสถูกไหม จำนวนตัวอย่างใกล้เคียงกันไหม และมีรูปหลุดผิดคลาสไหม กล่องสีน้ำเงินเป็นคำแนะนำของเว็บ ปิดได้

ขั้นที่ 3: เข้าใจตัวตั้งค่า 3 ตัว



ภาพหน้าจอจริง Advanced ของโปรเจกต์ Image

Epochs 50

จำนวนรอบที่โมเดลได้เห็นข้อมูลเทรนครบทุกตัวอย่าง

Batch Size 16

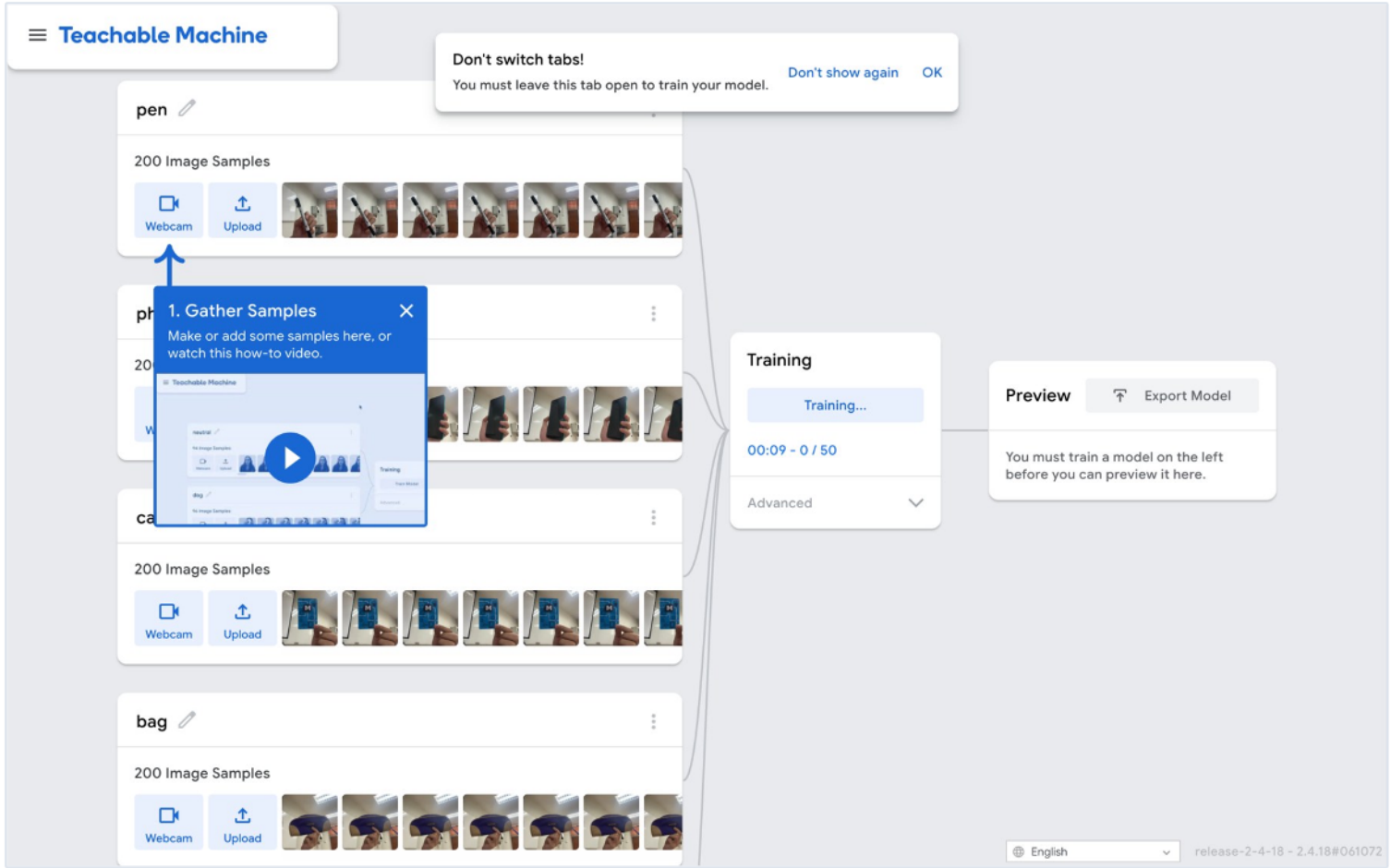
จำนวนตัวอย่างที่ใช้คำนวณก่อนปรับตัวเลขในโมเดล 1 ครั้ง

Learning Rate 0.01

ขนาดก้าวของการปรับแต่ละครั้ง มากไปอาจไม่นิ่ง น้อยไปเรียนช้า

ใน project.tm ของ repo: Image ใช้ Learning Rate 0.01 ส่วน Audio และ Pose ใช้ 0.001 ทั้ง 3 โปรเจกต์ใช้ Epochs 50 และ Batch Size 16 ผู้เริ่มต้นควรใช้ค่าที่ตั้งไว้ก่อน แล้วปรับข้อมูลเป็นอันดับแรก

ขั้นที่ 4: กด Train Model แล้วรอ



ภาพหน้าจอจริง ภาพระหว่างการเทรนจริงในการจัดทำเอกสารนี้

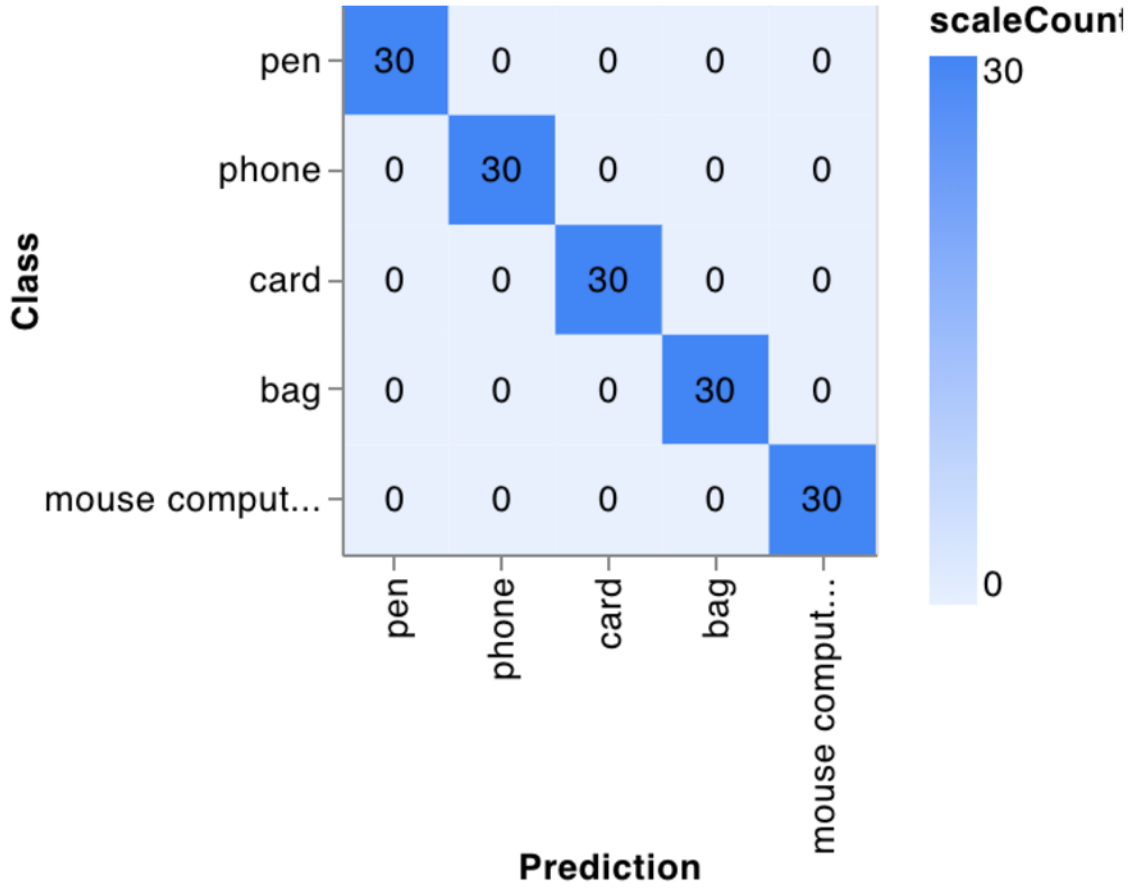
เกิดอะไรขึ้นตอนนี้

- เบราว์เซอร์คำนวณการเทรนในแท็บนี้
- ตัวนับ 0 / 50 คือจำนวน epoch ที่ทำไปแล้ว
- อย่าสลับแท็บ ตามคำเตือน “Don't switch tabs!”

ผลในการทดสอบนี้

โปรเจกต์ Image ซึ่งมี 1,000 รูป เทรนครบ 50 epoch ภายในไม่ถึง 1 นาที เวลาจริงจะต่างกันตามเครื่อง

ผลการเทรนจริง: Confusion Matrix



ผลจริง Under the hood หลังเทรน Image-Project/project.tm ด้านบนของกราฟถูกตัดโดยหน้าเว็บ

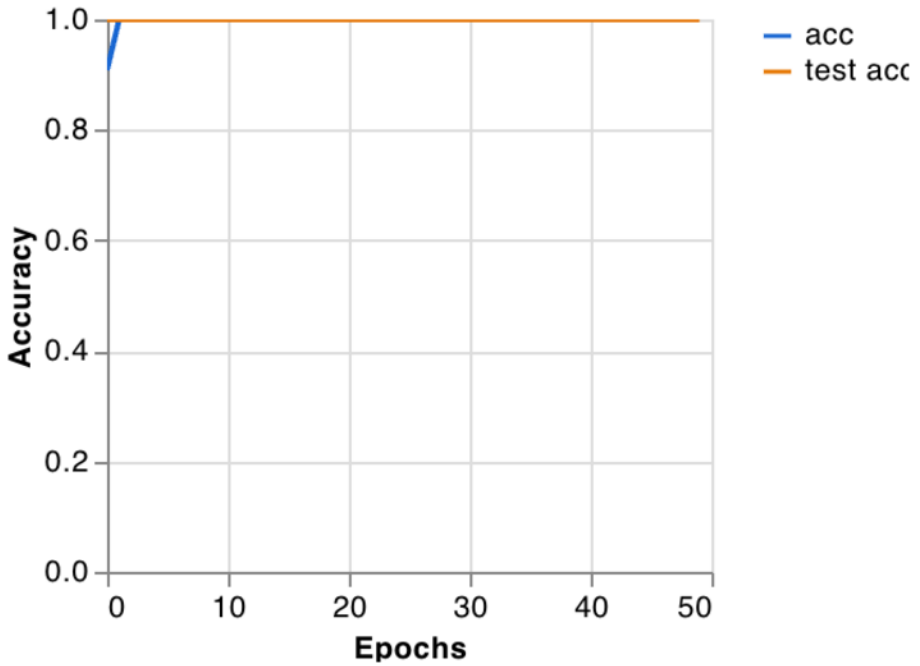
อ่านตารางนี้ยังไง

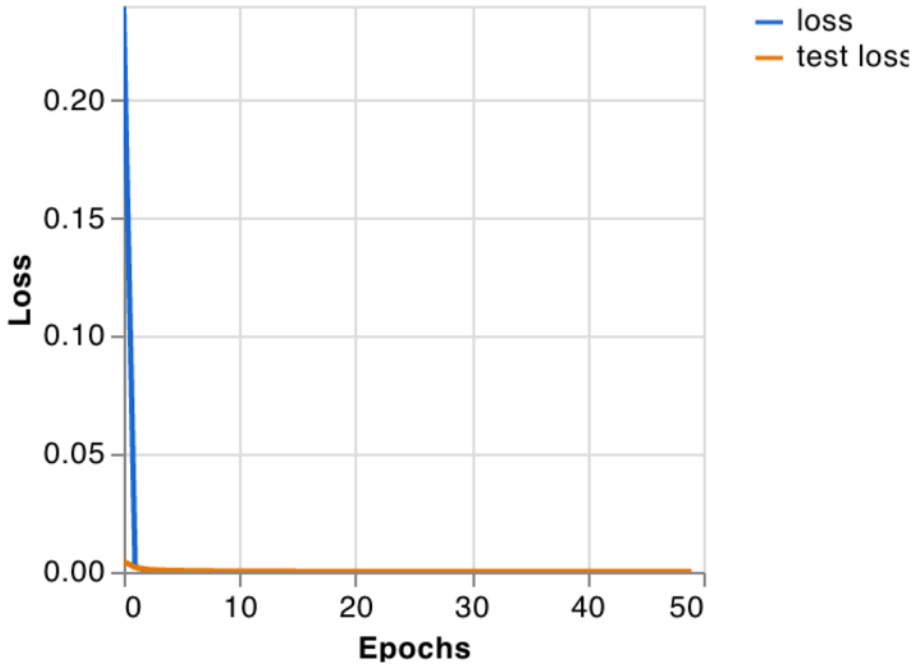
- แถว (Class) คือคำตอบจริง คอลัมน์ (Prediction) คือสิ่งที่โมเดลทาย
- เลขบนเส้นทแยงคือทายถูก เลขนอกเส้นทแยงคือทายผิด
- ตารางนี้ทายถูกทุกคลาส 30 จาก 30 และไม่มีการทายสลับคลาส

คลาส	Accuracy	จำนวนรูปสอบ
pen	1.00	30
phone	1.00	30
card	1.00	30
bag	1.00	30
mouse computer	1.00	30

Accuracy 1.00 คือถูก 100% ของรูปสอบ 30 รูป

ผลการเทรนจริง: กราฟระหว่างเรียน





Accuracy

ส่วนที่ทายถูก ยิ่งใกล้ 1 ยิ่งดี

Loss

ระดับความผิดพลาดที่โมเดลพยายามลด ยิ่งใกล้ 0 ยิ่งดี

ผลจริง Accuracy per epoch และ Loss per epoch จากหน้า Under the hood

สิ่งที่อ่านจากกราฟนี้

เส้นน้ำเงินคือกองเรียน เส้นส้มคือกองสอบ ทั้งสองเส้นถึงจุดดีที่สุดภายในไม่กี่ epoch แสดงว่าโจทย์นี้ง่ายมากสำหรับโมเดล เพราะรูปในกองสอบคล้ายกับกองเรียน ไม่ได้แปลว่าโมเดลจะเก่งเท่านี้ในห้องอื่นโดยอัตโนมัติ

คะแนน 100% ยังไม่ใช่เส้นชัย

Underfit

เรียนไม่พอ ทำได้ไม่ดีแม้กับข้อมูลเทรน
สังเกต: Accuracy ทั้งสองเส้นต่ำ
แก้: เพิ่มข้อมูลหรือจำนวนรอบ

พอดี

ทำได้ดีทั้งกองเรียนและกองสอบ ที่ควรหา
แต่ต้องทดสอบในสภาพจริงด้วย
สังเกต: สองเส้นดีและใกล้กัน

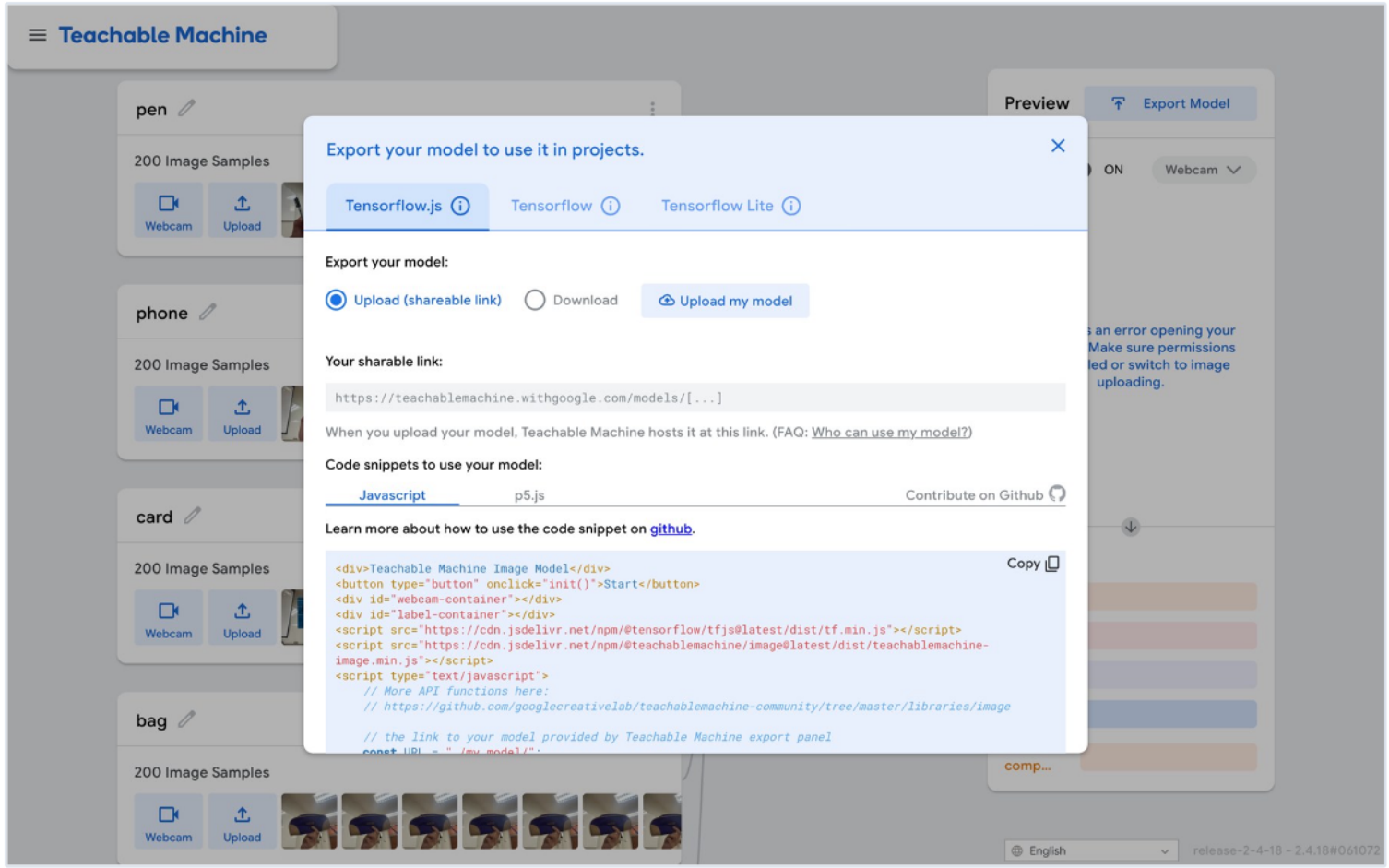
Overfit

จำข้อมูลเทรนแม่นเกินไป แต่ตอบกองสอบ
ไม่ดี
สังเกต: เส้นกองเรียนดีกว่ากองสอบมาก
แก้: เพิ่มข้อมูลที่หลากหลาย

กรณีของเรา: ทำไมจึงยังไม่ควรรีบสรุป

- กองสอบ 30 รูปมาจากการถ่ายชุดเดียวกับกองเรียน รูปจึงคล้ายกันมาก
- Data-Test ที่ได้ 5/5 มีแค่ 5 รูป ซึ่งน้อยเกินกว่าจะสรุปความแม่นยำ
- สิ่งที่ควรทำต่อ: ถ่ายรูปเพิ่มในห้องอื่น แสงอื่น และพื้นหลังอื่น แล้วทดสอบใหม่

ขั้นที่ 5: Export Model



ภาพหน้าจอจริง กดปุ่ม Export Model หลังเทรนเสร็จ

เก็บ project.tm ด้วยเมนู ≡ → Download project as file เพื่อกลับมาเทรนต่อครั้งหน้า

เลือกแบบไหนดี

Download

ได้ไฟล์ zip เก็บไว้ในเครื่อง โปรเจกต์นี้ใช้วิธีนี้
แตกไฟล์ลง application/my-model

Upload (shareable link)

ได้ลิงก์ไปวางในแอปได้ทันที แต่โมเดลจะถูกฝาก
ไว้บนบริการของ Teachable Machine
คิดก่อนว่าเหมาะกับงานนั้นไหม

บทที่ 4

ข้างในโมเดล

เปิดไฟล์ดูจริง ๆ ว่าโมเดลแต่ละงานต่างกันตรงไหน และตัวเลขเปลี่ยนเป็นคำตอบได้อย่างไร

ไฟล์โมเดลมี 3 ไฟล์ แต่ละไฟล์มีหน้าที่ต่างกัน

`model.json`

พิมพ์เขียว

บอกว่าโมเดลมีกี่ชั้น แต่ละชั้นเป็นแบบไหน และน้ำหนักมีรูปร่างเท่าไร

`weights.bin`

ความรู้ที่เรียนมา

ตัวเลขทศนิยมต่อกันเป็นไฟล์เดียว ได้จากการเทรน ชื่อรุ่นใหม่อาจเป็น model.weights.bin

`metadata.json`

ป้ายชื่อ

ชื่อคลาสตามลำดับ และชนิดของโมเดล แอปอ่านชื่อคลาสจากไฟล์นี้

โมเดลใน repo	model.json	weights.bin	จำนวนตัวเลขที่เรียนหรือเก็บไว้
Image	91,816 ไบต์	2,155,232 ไบต์	538,808 ค่า
Audio	5,997 ไบต์	5,754,676 ไบต์	1,438,669 ค่า
Pose	1,435 ไบต์	5,898,000 ไบต์	1,474,500 ค่า (ไม่รวม PoseNet)

นับจาก application/my-model ของแต่ละโปรเจกต์ โดยรวมค่าใน weightsManifest

Image: ใช้โมเดลที่เก่งอยู่แล้วมาสอนต่อ

แนวคิดนี้เรียกว่า Transfer Learning

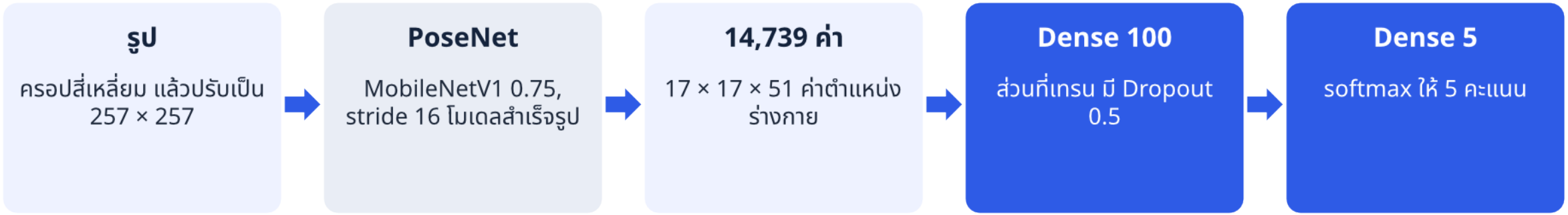
- เปรียบเหมือนคนที่ดูของเป็นอยู่แล้ว แค่บอกชื่อของใหม่ไม่กี่อย่างก็เรียนได้เร็ว เราจึงใช้ข้อมูลไม่มาก และเทรนในเบราว์เซอร์ได้
- model.json ของ repo มีชั้นชื่อ expanded_conv และ out_relu ซึ่งเป็นแบบแผนของ MobileNetV2 ตามด้วย GlobalAveragePooling2D
- ชั้น Dense สุดท้ายมี 5 ค่าเพราะมี 5 คลาส ถ้าเพิ่มคลาสแล้วเทรนใหม่ จำนวนนี้จะเปลี่ยนตาม

Audio: โมเดลอ่านสเปกโตรแกรมเหมือนอ่านรูป

แต่ละชั้นทำอะไร

ชนิดชั้น	เปรียบเทียบให้เห็นภาพ
Conv2D	แว่นขยายเล็ก ๆ ที่ไล่หาลวดลาย เช่น แถบความถี่หรือจังหวะเริ่มต้นของเสียง
MaxPooling2D	ย่อตาราง เก็บเฉพาะสัญญาณเด่น เพื่อลดขนาดข้อมูล
Dropout	ปิดบางส่วนแบบสุ่มตอนเทรน เพื่อไม่ให้โมเดลจำแม่นเกินไป
Dense + softmax	รวมทุกอย่างแล้วเปลี่ยนเป็นคะแนนของแต่ละคลาส

Pose: โมเดล 2 ตัวทำงานต่อกัน



ทำไม 14,739 ค่า

PoseNet แบ่งภาพเป็นตาราง 17 × 17 ช่อง แต่ละช่องมี 51 ค่า คือ ความน่าจะเป็นของ 17 จุด และระยะปรับตำแหน่งอีก 34 ค่า รวม 17 × 17 × 51 = 14,739 ค่า

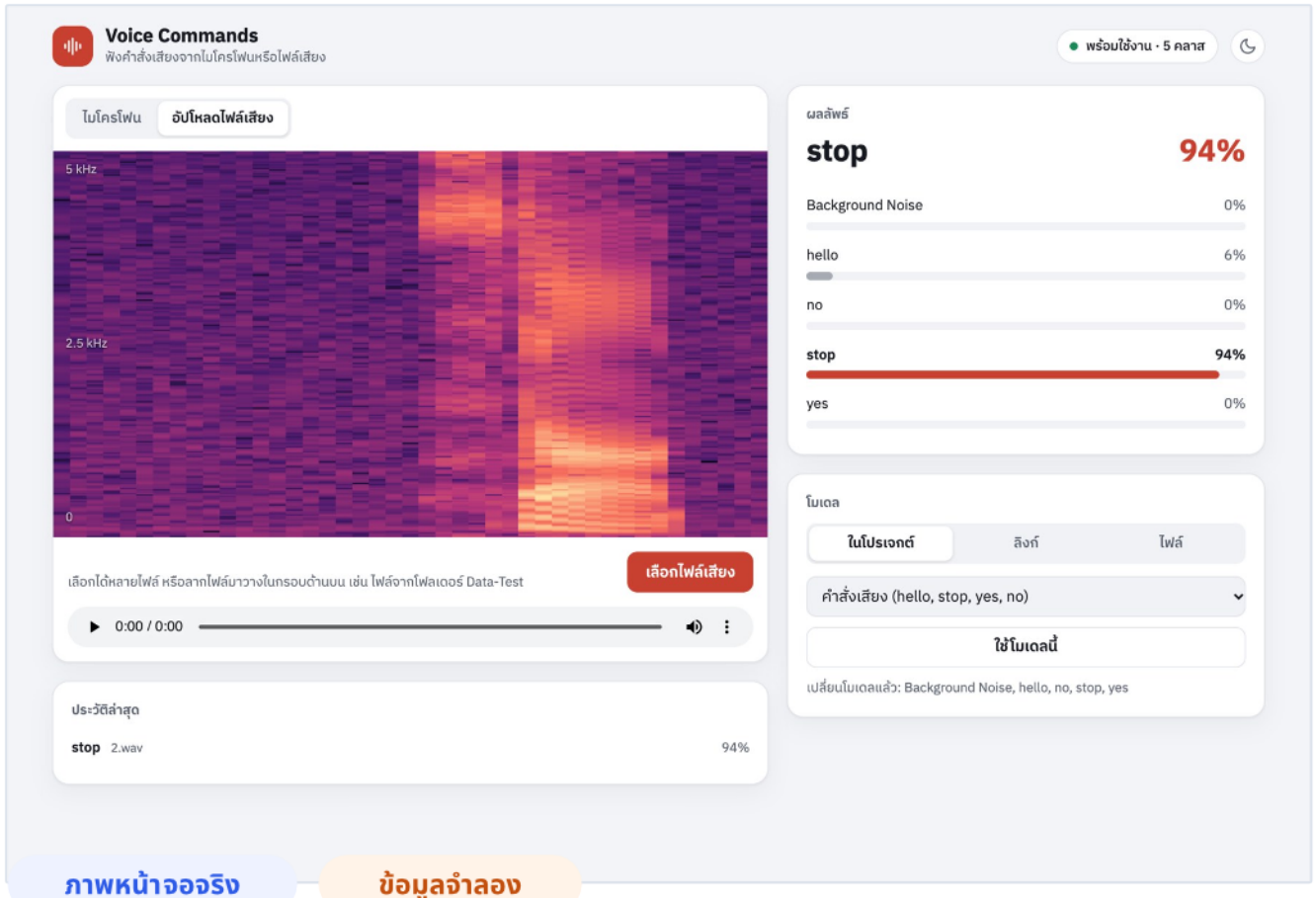
ข้อดี

โมเดลของเราเรียนจากข้อมูลตำแหน่ง จึงเรียนรู้ท่าทางจากข้อมูลที่กระชับกว่าทั้งรูป

ข้อจำกัด

ถ้า PoseNet หาจุดผิด โมเดลชั้นถัดไปก็ได้ข้อมูลผิดไปด้วย เราจะเห็นกรณีจริงในบทที่ 6

Softmax: จากตัวเลขดิบเป็นความมั่นใจ



ภาพหน้าจอจริง ข้อมูลจำลอง

ผลจริงของไฟล์ 2.wav

คลาส	ความมั่นใจ
stop	93.7%
hello	5.8%
คลาสที่เหลือรวมกัน	0.5%

คำนวณด้วย run_model.py ค่าปัดเศษ

Softmax เปลี่ยนตัวเลขดิบให้เป็นค่าบวกที่รวมกันได้ 100% เราจึงเลือกคลาสที่คะแนนสูงสุดเป็นคำตอบ

เกณฑ์ขั้นต่ำ

ถ้าคะแนนสูงสุดต่ำกว่าเกณฑ์ แอปถือว่าไม่แน่ใจ ค่าเริ่มต้น: Image กับ Pose 60% และ Audio 75%

บทที่ 5

นำโมเดลไปใช้

เปิดแอปในเครื่อง ลองกับไฟล์จริง เปลี่ยนโมเดล และเรียกใช้ด้วย Python

ติดตั้งและเปิดแอปใน 4 คำสั่ง

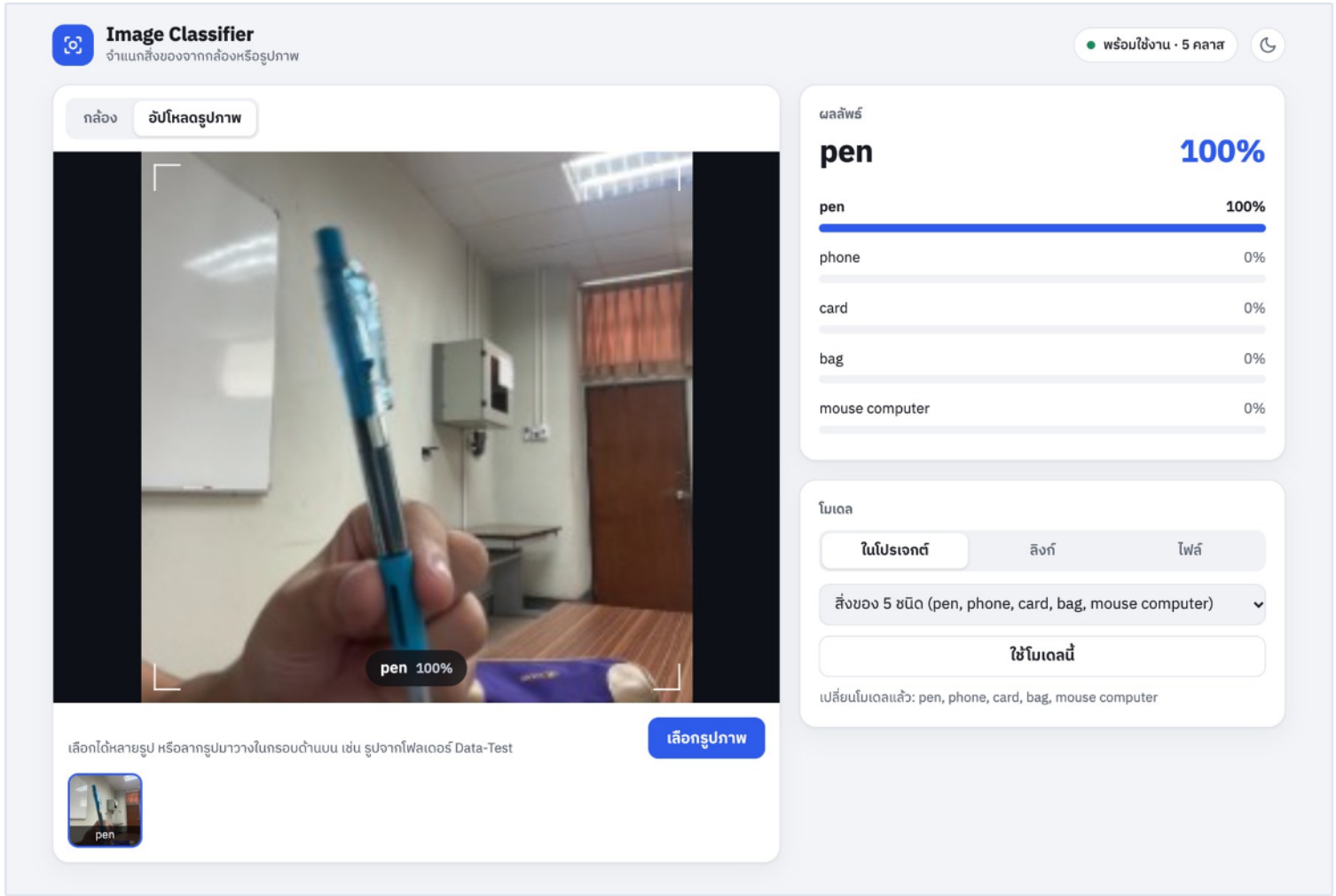
```
git clone https://github.com/bluebox-dev/Teachable-Machine-AnyThing.git
cd Teachable-Machine-AnyThing
bash install.sh
bash run.sh image
```

ระบบ	ติดตั้ง	เปิดแอป
macOS	bash install.sh	bash run.sh image หรือดับเบิลคลิก run.command
Linux	bash install.sh	bash run.sh image
Windows (Git Bash หรือ WSL)	bash install.sh	bash run.sh image
Windows (ไม่มี Git Bash)	ดับเบิลคลิก install.bat	ดับเบิลคลิก application\run.bat

เปลี่ยน image เป็น audio หรือ pose เพื่อเปิดแอปอื่น แอปใช้พอร์ต 8001, 8002 และ 8003 ตามลำดับ

install.sh และ run.sh ทดสอบแล้วบน macOS ส่วน Linux และ Windows ยังไม่ได้รันบนเครื่องจริง

แอป Image: ทดลองกับรูปจริง



ภาพหน้าจอจริง ข้อมูลจริง

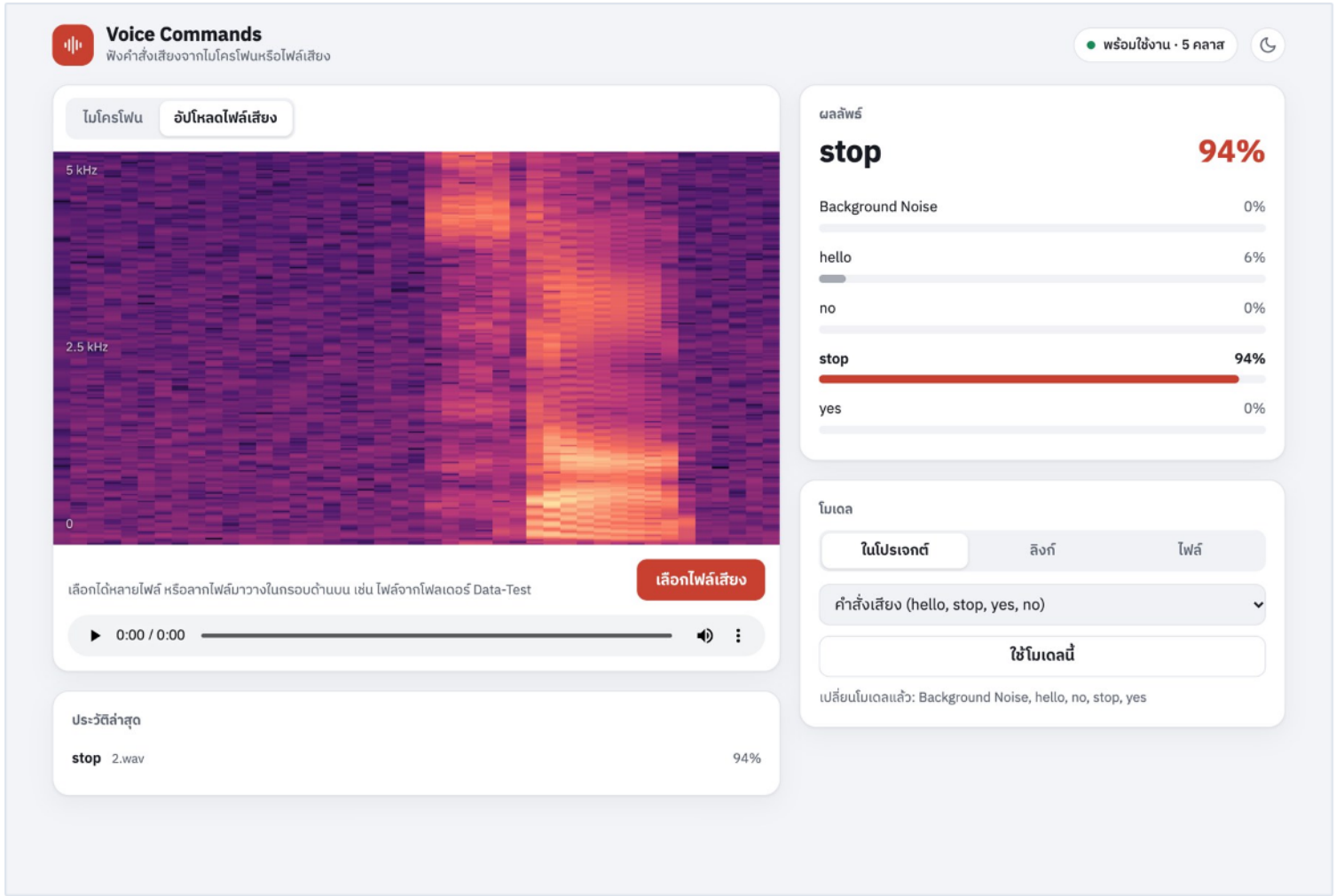
ลองทำ

1. กดแท็บ อัปโหลดรูปภาพ
2. เลือก Image-Project/Data-Test/3.jpg
3. ดูผลลัพธ์และแถบคะแนนด้านขวา

อ่านหน้าจอ

- มุมกรอบบอกพื้นที่ที่โมเดลเห็นจริง
- ผลลัพธ์แสดงคะแนนครบทุกคลาส
- เลือกได้หลายรูป แล้วกดดูย้อนหลังจากแถบล่าง
- ลองแท็บกล้องด้วยรูปหรือของจริงของคุณ

แอป Audio: ทดลองกับไฟล์เสียง



ภาพหน้าจอจริง ข้อมูลจำลอง

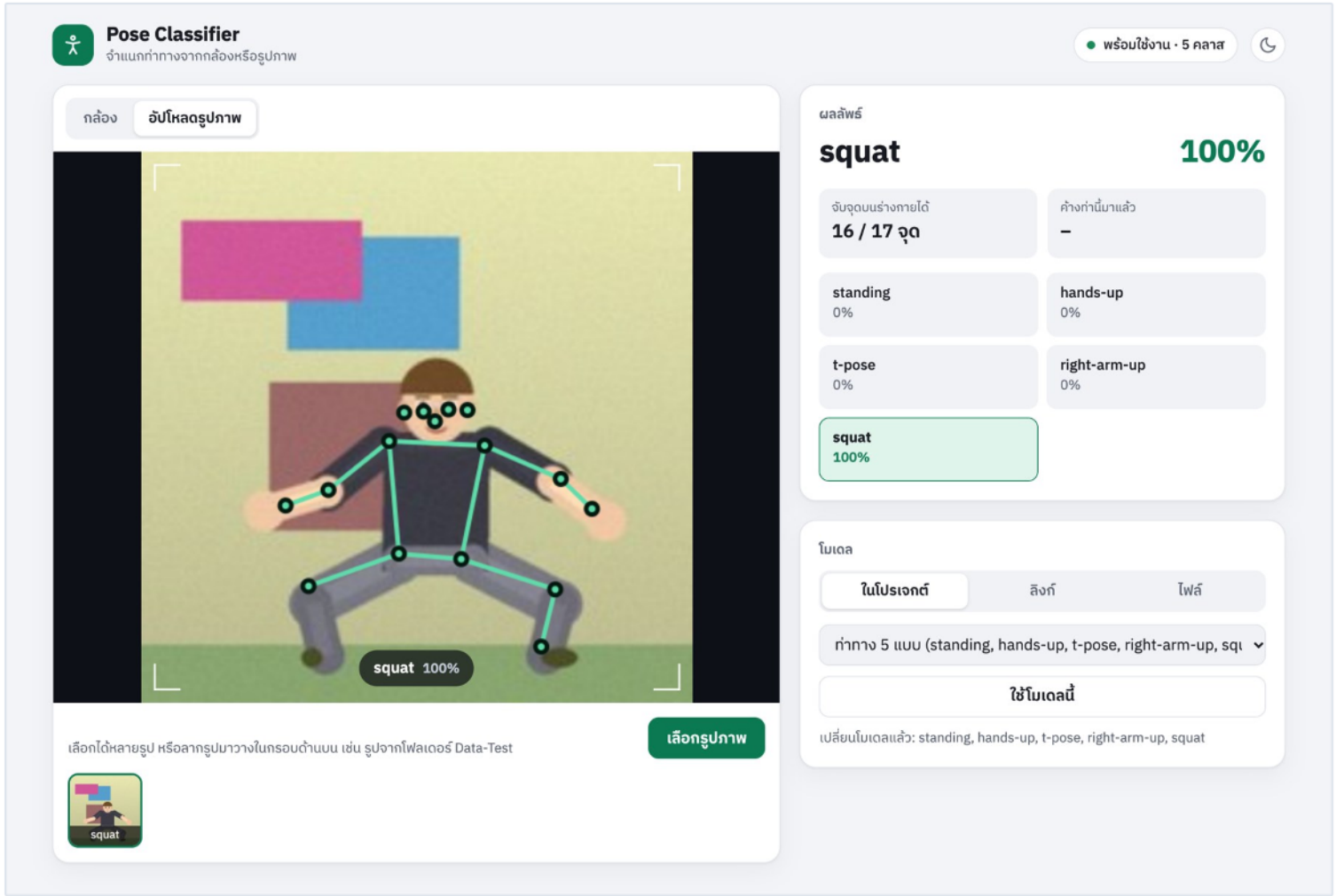
ลองทำ

1. เปิดด้วย bash run.sh audio
2. กดแท็บ อัปโหลดไฟล์เสียง
3. เลือก Audio-Project/Data-Test/2.wav

ตัวปรับในแอป

- ความมั่นใจขั้นต่ำ: คำที่คะแนนต่ำกว่านี้จะไม่ถูกนับ
- Overlap factor: ยิ่งมาก ยิ่งทำนายถี่
- โมเดลชุดนี้เรียนจากเสียงสังเคราะห์
  ถ้าจะใช้กับไมค์จริง ควรเก็บเสียงจริงแล้วเทรนใหม่

แอป Pose: ดูโครงร่างและผลท่าทาง



ภาพหน้าจอจริง ข้อมูลจำลอง

ลองทำ

1. เปิดด้วย bash run.sh pose
2. กดแท็บ อัปโหลดรูปภาพ
3. เลือก Pose-Project/Data-Test/5.jpg

อ่านหน้าจอ

- ภาพนี้ทำนาย squat 100%
- จับจุดบนร่างกายได้ 16 จาก 17 จุด
- เมื่อใช้กล้อง แอปจะจับเวลาว่าค้างท่าปัจจุบันนานเท่าไร

เปลี่ยนโมเดลได้ 3 วิธี โดยไม่ต้องแก้โค้ด

ในโปรเจกต์

เลือกโมเดลในรายการ เป็นค่าเริ่มต้นของแอป

ใช้ได้ทันที

ลิงก์

วางลิงก์จาก Teachable Machine แล้วโมเดลเปลี่ยนทันที และแอปจำลิงก์ล่าสุด

ต้องกด Upload my model ก่อน

ไฟล์

เลือก model.json, metadata.json และไฟล์ .bin จากเครื่อง

เหมาะกับโมเดลที่ไม่อยากฝากไว้ออนไลน์

เมื่อโมเดลเปลี่ยน ผลลัพธ์จะเปลี่ยนตาม

- แอปอ่านชื่อคลาสจาก metadata.json ของโมเดลใหม่ จึงสร้างแถบผลลัพธ์ตามจำนวนคลาสของโมเดลนั้นเอง
- ทดสอบแล้วกับโมเดลสาธารณะบน Teachable Machine ครบ 3 ชนิด ที่มี 3, 4 และ 2 คลาส
- ถ้าเลือกโมเดลผิดชนิด เช่น นำโมเดล Pose ไปใส่แอป Image แอปจะแจ้งเตือน

Python: ใช้ไฟล์โมเดลชุดเดียวกัน โดยไม่ต้องลง TensorFlow

```
bash run.sh image python --source Image-Project/Data-Test

โหลดโมเดลแล้ว: pen, phone, card, bag, mouse computer
1.jpg: card 100.0%
2.jpg: mouse computer 100.0%
3.jpg: pen 100.0%
4.jpg: phone 100.0%
5.jpg: bag 100.0%
```

ผลจริง ตัดบรรทัด “ตรวจพบ” ออกเพื่อให้อ่านง่าย

run_model.py ทำอะไร

- อ่าน model.json และ weights.bin เอง
- คำนวณทีละเลเยอร์ด้วย NumPy
- ไม่ใส่ --source จะเปิดกล้องหรือไมค์
- ปรับค่าได้ที่ส่วน ตั้งค่า ต้นไฟล์

โมเดล	เวลาต่อการทำนาย
Image	ราว 19 ms
Pose	ราว 30 ms
Audio	ราว 2 ms

ความเร็วนี้นับอะไร

ค่าจาก README วัดบน Apple M2, Python 3.9 และ NumPy 2.0 นับเฉพาะการคำนวณของโมเดล ไม่รวมการอ่านกล้องและการวาดภาพ

ต่อยอด: on_result คือจุดที่ AI เชื่อมกับงานของคุณ

```
ACTIONS = {
    "hello": lambda: print("สวัสดีครับ"),
    "stop": lambda: print("หยุดการทำงาน"),
}


def on_result(label, confidence):
    action = ACTIONS.get(label)
    if action:
        action()
```

ตัวอย่างจาก README สำหรับ Audio-Project/application/run_model.py

โค้ดนี้ทำงานอย่างไร

- ฟังก์ชันถูกเรียกเมื่อความมั่นใจเกิน MIN_CONFIDENCE
- label คือชื่อคลาส confidence คือความมั่นใจ
- แทนคำสั่ง print ด้วยงานของคุณได้

ไอเดียจาก README

- Image: บันทึกผลลงไฟล์ CSV
- Audio: สั่งงานตามคำที่ได้ยิน
- Pose: นับท่า และอ่านตำแหน่งข้อมือ

เมื่อ AI จะไปสั่งงานจริง

คิดเผื่อกรณีที่โมเดลตอบผิดเสมอ โดยเฉพาะงานที่มีผลต่อคนหรืออุปกรณ์ ควรมีเกณฑ์ความมั่นใจ และมีวิธีหยุดหรือยืนยันโดยคน

บทที่ 6

วัดผลอย่างซื่อสัตย์

คะแนนบอกอะไรได้ บอกอะไรไม่ได้ และโมเดลผิดอย่างไรเมื่อเจอข้อสอบจริง

ผลทดสอบจริงจาก Data-Test

Image

5/5

ถูกทุกรูป แต่ชุดทดสอบมีเพียง 5 รูป

Audio

9/10

ทาย hello เป็น no ไป 1 ไฟล์

Pose

8/10

right-arm-up ผิดทั้ง 2 รูป

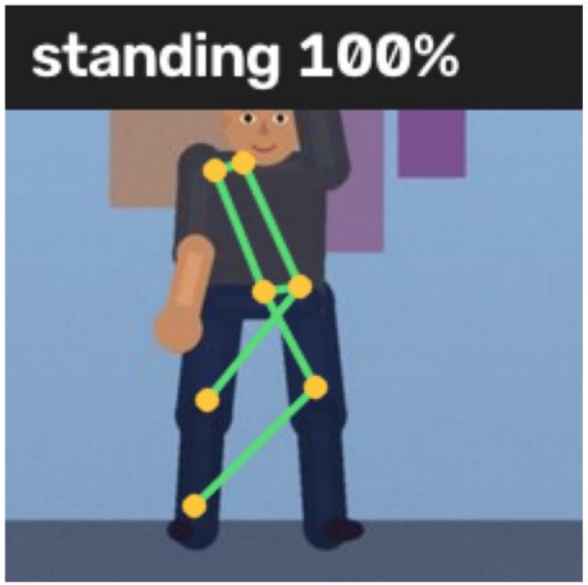
ไฟล์ที่ทายผิดหรือไม่มั่นใจมาก	คำตอบจริง	โมเดลทาย	ความมั่นใจ	ผล
Audio 6.wav	hello	no	86.8%	ผิด
Audio 9.wav	no	no	76.2% (อันดับ 2 คือ yes 20.1%)	ถูก แต่เฉียดเกณฑ์
Pose 4.jpg	right-arm-up	standing	100%	ผิด
Pose 9.jpg	right-arm-up	standing	100%	ผิด

ผลจริง รันด้วย run_model.py ในวันจัดทำเอกสาร ผลตรงกับตารางใน README

ตัวเลขเหล่านี้วัดกับข้อมูลใน repo เท่านั้น ยังไม่สะท้อนความแม่นกับเสียงหรือคนจริงของ Audio และ Pose

กรณีศึกษา: ทำไมมั่นใจ 100% แต่ยังผิด

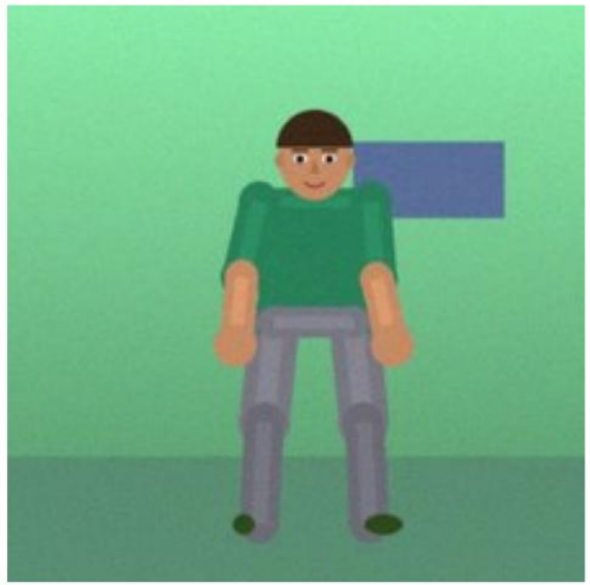
4.jpg คำตอบจริง right-arm-up



9.jpg คำตอบจริง right-arm-up

ภาพเทรนคลาส right-arm-up

ภาพเทรนคลาส standing



ข้อมูลจำลอง สองภาพซ้าย: ผลจริงจาก run_model.py สองภาพขวา: ตัวอย่างจากข้อมูลเทรน

หลักฐานจากโมเดล

- 4.jpg: จุด rightWrist ที่ PoseNet ส่งให้มีความมั่นใจเพียง 17%
- 9.jpg: PoseNet วาง rightWrist มั่นใจ 88% แต่อยู่ต่ำใกล้สะโพก ไม่ใช่เหนือศีรษะ

บทเรียน

- แขนที่ยกขึ้นคือหลักฐานสำคัญของท่านี้ เมื่อหาจุดนี้ไม่ได้ ท่าจึงน่าจะดูคล้าย standing
- ความมั่นใจคือคะแนนภายในโมเดล ไม่ใช่การรับรองว่าถูก

ก่อนบอกว่าโมเดล “ใช้ได้” ให้ถาม 5 ข้อ

คำถาม	กรณีในโปรเจกต์นี้
ข้อสอบมีจำนวนพอไหม	Image มี 5 รูป และ Audio กับ Pose มีอย่างละ 10 ไฟล์ จึงยังเล็ก
ข้อสอบเหมือนการใช้งานจริงไหม	Audio และ Pose ยังเป็นข้อมูลจำลอง
ผิดแล้วผิดแบบไหน	ผิดกระจุกที่ right-arm-up ควรเก็บตัวอย่างคลาสนี้เพิ่ม
ทดสอบบนอุปกรณ์จริงแล้วหรือยัง	เว็บทดสอบด้วยสตรีมจำลอง ยังไม่ได้ลองกล้องและไมค์จริง
ผิดแล้วเกิดผลอะไร	สำหรับการเรียนรู้ผลไม่รุนแรง แต่ถ้าสั่งงานจริงต้องมีวิธีป้องกัน

สรุปสถานะ: โปรเจกต์นี้เหมาะสำหรับเรียนรู้และทดลอง ยังไม่ใช่ระบบที่พร้อมนำไปใช้จริง

บทที่ 7

ฝึกปฏิบัติ

ลงมือทำ 3 แล็บ แล้วทบทวนด้วยแบบทดสอบ

แล็บ 1: ลองทำให้โมเดลตอบผิด

เป้าหมาย

เห็นด้วยตาว่าโมเดลเก่งตรงไหน และพังตรงไหน

ขั้นตอน

1. รัน bash run.sh image แล้วอัปโหลด Data-Test ทั้ง 5 รูป จดผลลัพธ์
2. ถ่ายรูปปากกาของคุณเองในห้องอื่น ถ้าเป็นไปได้ให้พื้นหลังต่างจากชุดเทรน แล้วอัปโหลด
3. ถ่ายรูปของที่ไม่อยู่ใน 5 คลาส เช่น แก้วน้ำ แล้วดูว่าโมเดลตอบอะไร
4. บันทึกผลลงตารางด้านล่าง

รูปที่ลอง	คำตอบที่ควรเป็น	โมเดลตอบ	ความมั่นใจ	ถูกไหม

คำถามชวนคิด: ถ้าเจอของที่ไม่อยู่ในคลาส โมเดลตอบว่า “ไม่รู้” ได้ไหม เพราะอะไร

แล็บ 2: เทรนโมเดลของคุณเอง

เป้าหมาย

เดินครบวงจร 5 ขั้นด้วยข้อมูลของคุณเอง โดยใช้ 2–3 คลาส เช่น แก้วน้ำ กับ กุญแจ

1. เก็บ

เปิด Image Project แล้วเก็บคลาสละ 50 รูปขึ้นไปจากกล้อง หมุนและขยับของหลายมุม

2. เทรน

กด Train Model ใช้ค่าเริ่มต้น และเปิดแท็บค้างไว้

3. ทดสอบ

ใช้ Preview และเปิด Under the hood เพื่ออ่าน confusion matrix

4. Export

เลือก Download หรือ Upload ตามความเหมาะสม และบันทึก project.tm

5. ใช้งาน

ลองวางลิงก์หรือไฟล์ในแอป แล้วตรวจว่าแถบคลาสเปลี่ยนตาม

เกณฑ์ว่าแล็บนี้สำเร็จ

- โมเดลแยกคลาสได้ในมุมใหม่ที่ไม่เคยเก็บ
- อธิบายได้ว่าคลาสไหนยังผิด และจะเพิ่มข้อมูลแบบไหน
- คุณเปิดแอปและเห็นชื่อคลาสของคุณเองในผลลัพธ์

แล็บ 3: ทำให้โมเดลไปทำงานให้คุณ

เป้าหมาย

ใช้ on_result บันทึกผลทำนายลงไฟล์ CSV เป็นการต่อยอดที่พบบ่อยและเขียนไม่ยาก

```
import csv
from datetime import datetime


def on_result(label, confidence):
    with open("results.csv", "a", newline="", encoding="utf-8") as file:
        csv.writer(file).writerow([datetime.now().isoformat(timespec="seconds"), label, f"{confidence:.3f}"])
```

ตัวอย่างจาก README แทนที่ฟังก์ชัน on_result เดิมใน Image-Project/application/run_model.py ของสำเนาที่คุณใช้ฝึก

ขั้นตอน

1. ทำสำเนาโฟลเดอร์ Image-Project ไว้ทดลอง แล้วแก้ฟังก์ชันในสำเนา
2. รัน Python จากสำเนานั้นด้วย --source ../Data-Test
3. เปิด results.csv ดูผล แล้วลองปรับ MIN_CONFIDENCE เพื่อเทียบจำนวนแถว

แบบทดสอบท้ายบท

1. โมเดลต่างจากโปรแกรมที่เขียนกฎเองอย่างไร
2. ทำไมต้องแยก Training กับ Test
3. สเปกโตรแกรมแสดงอะไรบ้าง 3 อย่าง
4. Epoch 50 หมายความว่าอะไร
5. เลขนอกเส้นทแยงของ confusion matrix แปลว่าอะไร
6. ความมั่นใจ 100% แปลว่าถูกแน่นอนหรือไม่ ยกตัวอย่างจากบทที่ 6
7. ไฟล์ใดเก็บชื่อคลาส และไฟล์ใดเก็บน้ำหนัก
8. ถ้าจะใช้โมเดล Pose กับคนจริง ควรทำอะไรก่อน

เฉลยแบบทดสอบ

1. โปรแกรมทำตามกฎที่คนเขียน โมเดลหารูปแบบเองจากตัวอย่าง
2. เพื่อวัดผลกับข้อมูลที่โมเดลไม่เคยเรียน คะแนนจึงเชื่อถือได้กว่า
3. เวลา (แกนนอน) ความถี่ (แกนตั้ง) และความดัง (ความสว่างของสี)
4. โมเดลได้เห็นตัวอย่างในกองเทรนครบทุกตัว 50 รอบ
5. โมเดลทายผิด โดยทายเป็นคลาสตามคอลัมน์นั้น
6. ไม่ ภาพ Pose 4.jpg และ 9.jpg ถูกทาย standing 100% ทั้งที่คำตอบจริงคือ right-arm-up
7. metadata.json เก็บชื่อคลาส และ weights.bin เก็บน้ำหนัก
8. เก็บตัวอย่างจริงจากกล้องในสถานที่จริง แล้วเทรนใหม่และทดสอบใหม่

อภิธานศัพท์

คำ	ความหมาย	คำ	ความหมาย
Accuracy	สัดส่วนที่ทายถูก	MobileNet	โครงข่ายภาพขนาดเล็กที่เหมาะกับอุปกรณ์ทั่วไป
Batch Size	จำนวนตัวอย่างต่อการปรับหนึ่งครั้ง	Overfit	จำข้อมูลเทรนได้ แต่ไม่เก่งข้อมูลใหม่
Confusion Matrix	ตารางคำตอบจริงเทียบกับที่ทาย	PoseNet	โมเดลหาตำแหน่ง 17 จุดบนร่างกาย
Dense	ชั้นที่รวมค่าทั้งหมดเข้าด้วยกัน	Softmax	แปลงค่าเป็นความมั่นใจรวม 100%
Epoch	การเห็นข้อมูลเทรนครบหนึ่งรอบ	Spectrogram	ภาพเสียงตามเวลาและความถี่
Learning Rate	ขนาดก้าวในการปรับโมเดล	Transfer Learning	สอนต่อจากโมเดลที่เรียนมาแล้ว
Loss	ค่าความผิดพลาดที่พยายามลด	Underfit	เรียนไม่พอ ทำได้ไม่ดีแม้ข้อมูลเทรน

แหล่งข้อมูลของเอกสารนี้

เนื้อหา	ที่มา
ชื่อคลาส จำนวนข้อมูล คำสั่ง ข้อจำกัด เวลาประมวลผล	README.md และไฟล์ในโปรเจกต์ Teachable-Machine-AnyThing
โครงสร้างโมเดลและขนาดไฟล์	application/my-model/ ของแต่ละโปรเจกต์
ค่า Epochs, Batch Size และ Learning Rate	manifest.json ภายใน project.tm
ผลทดสอบ คะแนนรายไฟล์ และจุดบนร่างกาย	รัน application/run_model.py กับ Data-Test
ภาพแอป	จับภาพแอปจริงด้วย Google Chrome และไฟล์จาก Data-Test
ภาพ Teachable Machine และผลการเทรน	ใช้งานจริงที่ teachablemachine.withgoogle.com รุ่น 2.4.18 ด้วย project.tm ของ repo
ซอฟต์แวร์เบื้องหลัง	Teachable Machine, TensorFlow.js, PoseNet, Speech Commands, NumPy, OpenCV

ผลการเทรนบทที่ 3 มาจากการเทรนซ้ำเพื่อจัดทำเอกสาร ไม่ได้เขียนทับโมเดลใน repo เอกสารนี้จัดทำจาก commit aa3b7b5